NETWORK · SECURITY · PRACTICE

OpenVPN
企业实战指南

从协议原理到架构设计，从零搭建到企业级安全部署
一本真正能落地的 OpenVPN 实战手册

协议原理 | 架构设计 | 证书体系 | 隧道封装 | 高可用 | 安全加固 | 监控运维 | 故障排查

完整深入版 · 图文并茂

适用读者：网络工程师 / 安全运维 / 系统管理员 / 技术爱好者

版本：v1.0（2026）

目 录

第一部分　基础与原理

01 VPN 与 OpenVPN 概述 3

02 OpenVPN 工作原理与协议栈 6

03 加密与认证机制 9

04 隧道封装与路由原理 12

第二部分　安装与配置

05 环境准备与安装 15

06 PKI 证书体系搭建 18

07 服务端配置详解 21

08 客户端配置详解 24

第三部分　进阶与企业实践

09 高级特性：双因素认证与脚本钩子 27

10 企业高可用与负载均衡 29

11 安全加固最佳实践 32

12 监控、日志与告警 35

13 故障排查实战 38

14 企业实战案例 40

附录

A 配置参数速查表 43

B 常用命令速查 45

PART 1 · 基础与原理

第1章　VPN 与 OpenVPN 概述

> 本章是全书的起点。我们先回答"VPN 到底是什么、解决什么问题"，再引出本书主角 OpenVPN，梳理它的历史、特点与适用边界，帮助你在动手之前建立正确的全局认知。

1.1　什么是 VPN

VPN（Virtual Private Network，虚拟专用网络）是一种在不可信的公共网络（通常是互联网）之上，通过加密与隧道技术构建出的逻辑上隔离、安全的私有通信通道。它让分布在不同物理位置的主机，看起来就像处于同一个私有局域网内一样互相通信。

"虚拟"意味着这条网络并非由一条真实铺设的专线构成；"专用"意味着它对外是不可见、不可窥探的。理解 VPN 的关键在于两个词：**隧道（Tunneling）**与**加密（Encryption）**。

> **隧道：**把一份完整的网络报文（内层）整体作为"数据载荷"，再封装进另一份报文（外层）里传输，到达对端后再解封装还原。 **加密：**对隧道内传输的数据进行加密与完整性校验，保证即使报文在公网被截获，攻击者也无法读取内容或篡改。

1.2　VPN 的两种典型形态

从应用场景看，VPN 主要分为两大类，这也是理解 OpenVPN 架构的基石：

形态	英文	典型场景	参与方
远程接入 VPN	Remote-Access VPN	员工在家/出差访问公司内网	一个服务端 + 大量移动客户端
站点到站点 VPN	Site-to-Site VPN	总部与分支机构、多机房互联	两个（或多个）固定网关互连

OpenVPN　对这两种形态都支持得很好：它既能为单台电脑提供远程接入，也能让两台网关设备之间建立长期稳定的站点互联隧道。本书第二部分会分别给出两种形态的完整配置。

1.3 OpenVPN 的历史与特点

OpenVPN 由 **James Yonan** 于 2001 年发布，采用 **GPLv2** 开源协议，是一个完全基于用户态实现的 SSL/TLS VPN 解决方案。二十多年来，它凭借出色的稳定性和灵活性，成为部署最广泛的开源 VPN 软件之一。

OpenVPN 的核心特点可以概括为：

- **基于成熟的 SSL/TLS 协议：**复用 OpenSSL 库完成认证、密钥协商与加密，安全性经过长期验证。
- **UDP/TCP 双模式：**默认使用 UDP 以获得更高性能，也可切换为 TCP（如 443 端口）以增强穿透性。
- **强 NAT 穿透能力：**无需特殊网络设备即可穿越大多数 NAT 与防火墙。
- **跨平台：**支持 Linux、Windows、macOS、BSD、Android、iOS，甚至路由器固件（如 OpenWrt）。
- **灵活可扩展：**支持证书认证、用户名密码、双因素认证、脚本钩子、每用户权限控制等丰富特性。
- **社区与商业版并存：**开源社区版（Community Edition）与商业产品 OpenVPN Access Server / Cloud 共享同一技术内核。

1.4 主流 VPN 方案对比

选择 VPN 方案前，先看清 OpenVPN 在整个技术版图中的位置，有助于做出正确决策。

方案	协议基础	默认端口	优势	劣势
OpenVPN	SSL/TLS	1194/UDP	跨平台、NAT 穿透强、生态成熟、功能丰富	用户态实现，极限性能略低
WireGuard	Noise 协议框架	51820/UDP	代码精简、内核态、性能极高	生态较新、动态地址管理需额外工具
IPsec（IKEv2）	标准协议族	500/4500/UDP	标准化、系统级支持	配置复杂、NAT 穿透较弱
SSTP	SSL/TLS	443/TCP	穿透强（伪装 HTTPS）	微软主导、跨平台一般
PPTP / L2TP	老旧协议	1723 等	兼容旧设备	PPTP 已被破解，不推荐

> **提示**
>
> 若追求极简与高性能且设备支持，可选 WireGuard；若看重跨平台兼容、企业级功能（证书体系、访问控制、审计）与长期稳定性，**OpenVPN 仍是企业场景最稳妥的选择**。

1.5 OpenVPN 的典型应用场景

- **远程办公接入：**员工通过公司下发的证书与配置文件，安全访问内网 OA、代码库、数据库。
- **多云 / 多机房互联：**打通云上 VPC 与本地 IDC，构建混合云网络。
- **分支机构互联：**让分散各地的办公室像在同一局域网。
- **安全运维通道：**为 SSH、数据库等敏感服务提供加密的专用管理入口。
- **物联网 / 边缘设备回连：**让现场设备安全回连中心平台。

1.6 OpenVPN 整体架构一瞥

下面这张架构图是全书内容的骨架，后续各章都会围绕它展开。请先建立整体印象：OpenVPN 由**客户端、服务器、证书颁发机构（CA）**三部分组成，通信分为**控制面**（TLS 握手）与**数据面**（加密隧道）两条线。

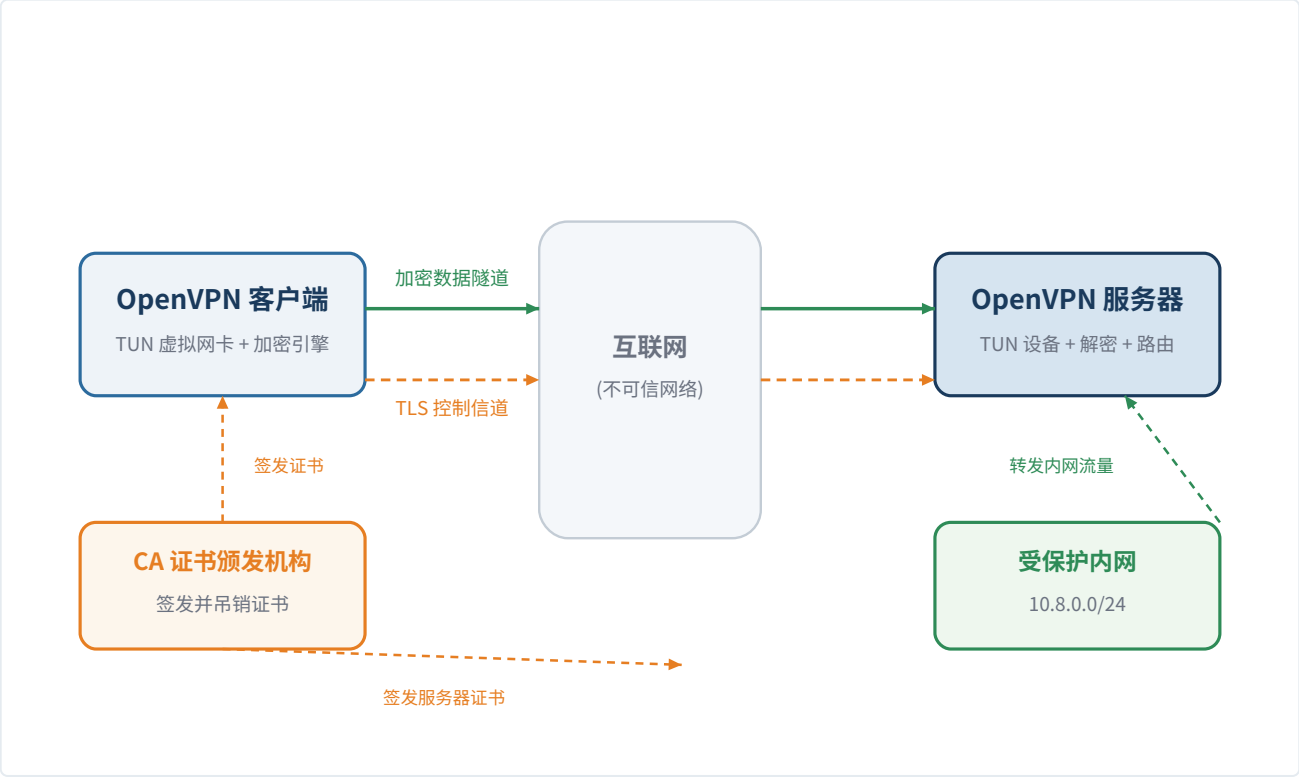


图 1-1 OpenVPN 整体架构：客户端、服务器与 CA 三者协作，控制面与数据面分离

在本章建立的整体框架下，下一章我们将深入 OpenVPN 的内部工作原理，拆解它的协议栈与数据流转过程。

PART 1 · 基础与原理

第2章　OpenVPN 工作原理与协议栈

> 理解原理是排障与调优的前提。本章拆解 OpenVPN 的客户端-服务器模型、控制面与数据面的分工、UDP 与 TCP 两种模式的取舍，并完整走一遍"一次连接的生命周期"。

2.1　客户端-服务器模型

OpenVPN 采用经典的**客户端-服务器（C/S）架构**。服务器端监听一个固定端口（默认 1194/UDP），等待客户端发起连接；客户端主动连接服务器，完成 TLS 认证后建立加密隧道。一台服务器可以同时服务成百上千个客户端，每个客户端在服务端表现为一个独立的"会话"。

与点对点协议（如 WireGuard 早期形态）不同，OpenVPN 的 C/S 模型天然适合**远程接入**场景：证书的签发与吊销都集中在服务端，管理员可以统一管控所有接入者。

2.2　控制面与数据面

OpenVPN 的通信被清晰地划分为两个平面，这是它设计的精髓：

平面	职责	使用的技术	发生时机
控制面	身份认证、密钥协商、参数协商、保活	TLS 1.2/1.3 握手	连接建立时 + 周期性重协商
数据面	业务数据的加密、封装、转发	对称加密（AES-GCM 等）	连接建立后的每一秒

控制面用非对称密码学（证书 + TLS）解决"我们是谁、用什么密钥通信"的问题，开销大但频率低；**数据面**用协商好的对称密钥对每个报文加解密，开销小但频率极高。这种"非对称认证 + 对称传输"的分层设计，是所有现代安全协议的共同范式。

2.3　通信协议栈

OpenVPN 的报文在网络上传输时遵循如下协议栈：

```
应用层    | OpenVPN 协议（控制消息 / 数据消息）
传输层    | TLS 1.3（控制面）   +   AES-GCM（数据面）
```

```
承载层    | UDP 1194（默认）  或  TCP 1194
网络层    | IP
```

UDP 模式是默认也是推荐模式：隧道内的 TCP 流量不必再承受"TCP over TCP"的重复重传放大效应，延迟更低、吞吐更高。

TCP 模式适用于受限网络：某些企业防火墙只放行 TCP，或 UDP 被运营商 QoS 严重限速时，用 TCP 443 端口可以把 VPN 流量伪装成普通 HTTPS，穿透性最好，代价是性能下降。

> **说明**
>
> TCP over TCP 问题：如果隧道用 TCP 承载，隧道内的 TCP 连接重传时会触发外层 TCP 也重传，形成"重传风暴"。除非万不得已，**生产环境请优先使用 UDP 模式**。

2.4 一次连接的生命周期

从客户端敲下"连接"按钮到能正常访问内网，整个过程分为五步：

1. **初始化：**客户端加载配置与证书，向服务端发起 UDP/TCP 连接。
2. **TLS 握手：**双方完成证书交换与验证，协商出对称会话密钥。
3. **授权校验：**服务端校验客户端证书是否由本 CA 签发、是否在有效期内、是否被吊销（CRL）；若配置了用户名密码或双因素，此步一并验证。
4. **虚拟 IP 分配与路由推送：**服务端从地址池为客户端分配虚拟 IP，并推送路由、DNS 等配置。
5. **数据传输与保活：**双方周期性交换 keepalive 报文，并在密钥生命周期到期时静默重协商。

其中第 2 步 TLS 握手是理解认证机制的关键，时序如下：

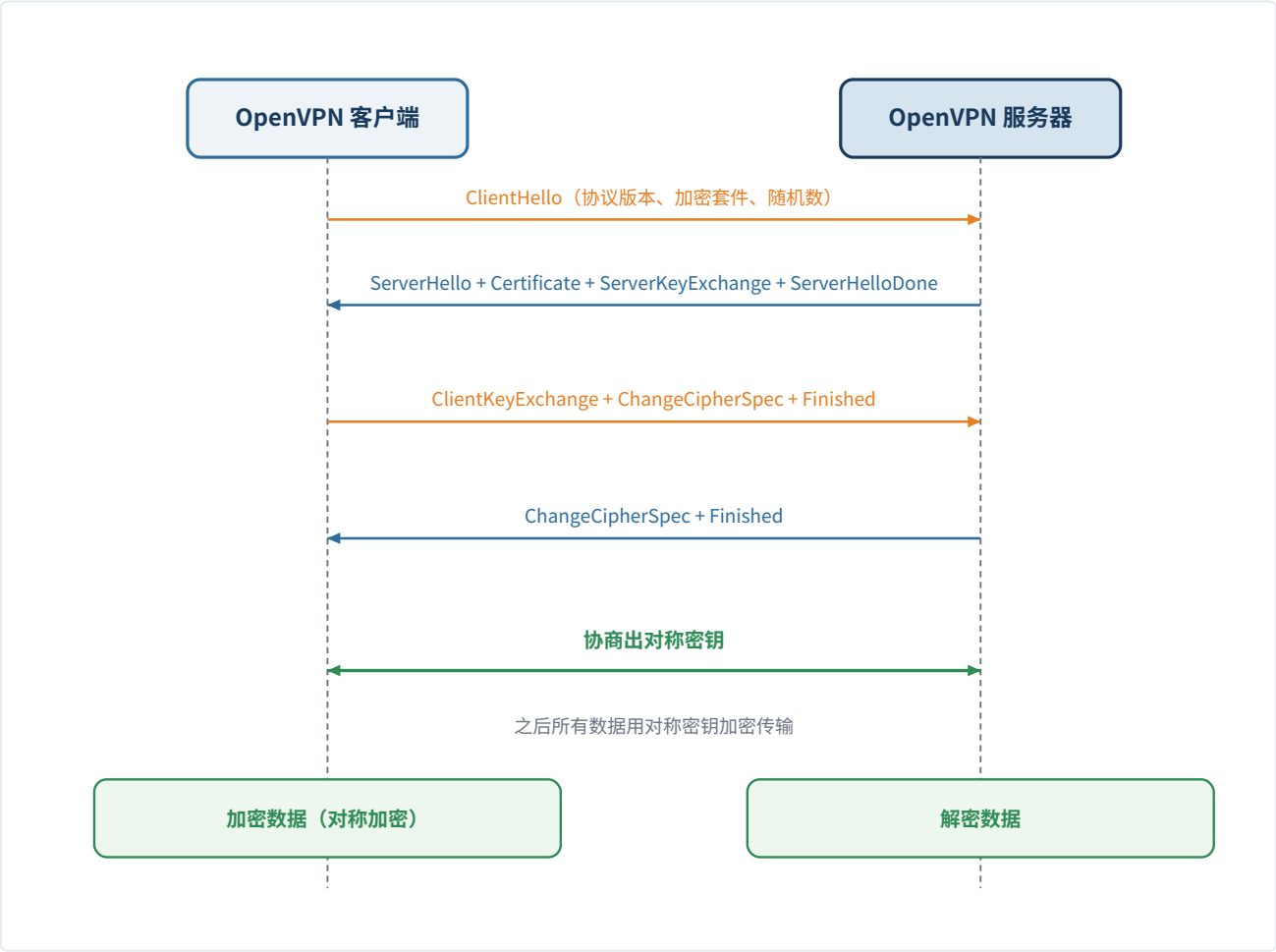


图 2-1　OpenVPN 控制面 TLS 握手时序：证书认证在前，对称密钥协商在后

2.5　进程模型与管理接口

OpenVPN 是**单进程、事件驱动**的架构：一个服务端进程通过 I/O 多路复用同时处理所有客户端会话，没有为每个客户端派生线程，因此内存占用小、上下文切换开销低。若配置了 `thread` 相关选项，加解密可卸载到工作线程（版本 2.5+ 支持多线程数据面）。

服务端还提供一个**管理接口（management interface）**，通过 Telnet 或 Unix Socket 暴露当前连接列表、流量统计，并支持踢出用户、发送消息等操作。这是第　12　章监控体系的数据来源，此处先埋个伏笔。

> **提示**
>
> 想亲眼看看这些机制？在测试机上执行 `openvpn --help` 浏览选项，或连接 management 接口执行 `status` 命令，输出会与本章的模型一一对应。原理不再神秘，排障就成功了一半。

PART 1 · 基础与原理

第3章　加密与认证机制

> "VPN　安全吗？"要回答这个问题，必须理解它用了哪些密码学工具、各自保护什么。本章用最平实的语言讲清 OpenVPN 的四层防线：对称加密、证书认证、HMAC 校验与前向保密。

3.1　数据面：对称加密

隧道里流动的每一个业务报文，都用**对称加密算法**保护。OpenVPN 现代版本默认使用 **AES-256-GCM**，它属于 AEAD（认证加密）算法——一次操作同时完成加密与完整性校验，无需再叠加单独的 HMAC，性能与安全性俱佳。

算法	类型	说明
AES-256-GCM	AEAD 对称加密	当前默认，硬件加速（AES-NI）下性能极好
AES-128-GCM	AEAD 对称加密	安全性足够，性能更高，适合低配设备
ChaCha20-Poly1305	AEAD 对称加密	无 AES 硬件加速的设备（如部分移动端）上更快
BF-CBC / DES	传统分组密码	**已不安全，新部署必须禁用**

3.2　控制面：证书与非对称密码学

对称密钥如何安全地送到对方手里？答案是**非对称密码学**：服务端持有私钥，客户端用服务端证书里的公钥加密密钥材料；反过来服务端也验证客户端证书。整个过程第三方无法窃听或伪造。

而"这把公钥真的属于服务器本人吗"由**数字证书**回答：证书是 CA（证书颁发机构）用自己私钥签署的身份声明。客户端只信任自家的 CA 证书，因此伪造的证书无法通过验证——这就是 PKI 体系的信任锚点。

3.3　PKI 与 CA：信任的根基

OpenVPN 的部署离不开一套 PKI（Public Key Infrastructure）。核心角色只有三个：

- **CA 根证书（ca.crt）：**信任的起点，服务端和所有客户端都要持有。
- **服务器证书：**证明"我是合法的服务端"，由 CA 签发。
- **客户端证书：**证明"我是合法的接入者"，每人一张，由同一 CA 签发。

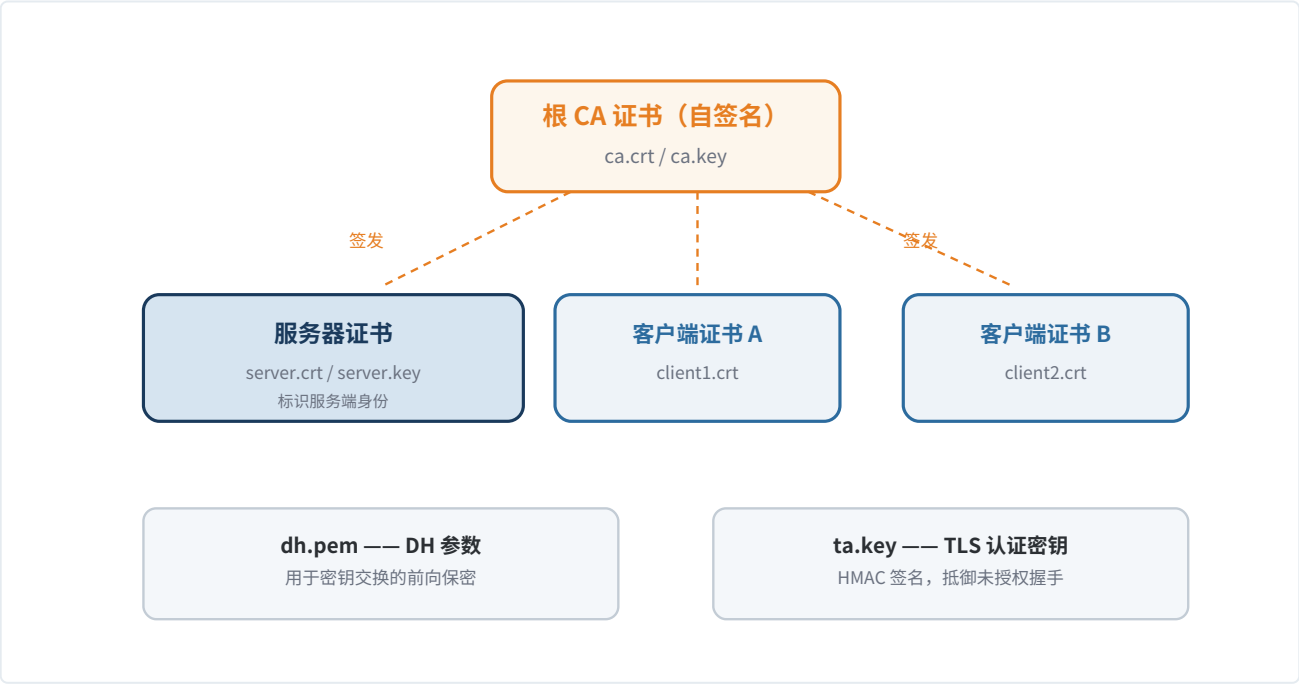


图 3-1 OpenVPN PKI 体系：一个 CA 向下签发服务器与客户端证书，辅以 DH 参数与 TLS 认证密钥

CA 私钥（ca.key）是整个体系的命门：**谁拿到它，谁就能签发任意"合法"证书**。生产环境中它必须离线保管、严格授权，绝不能部署在 VPN 服务器上。第 6 章将手把手搭建这套体系。

3.4 HMAC 防线：tls-auth 与 tls-crypt

TLS 握手报文本身若不设防，攻击者可以向服务器发起海量伪造握手，耗尽资源（DoS）。OpenVPN 提供两道补充防线：

机制	原理	效果
tls-auth（ta.key）	对 TLS 控制报文附加 HMAC 签名	没有 ta.key 的报文直接丢弃，握手报文不加密
tls-crypt	用共享密钥对整个控制信道加密 + 认证	连握手内容也对外不可见，还可隐藏服务器指纹

企业部署推荐使用 **tls-crypt**：安全性更强，且避免了 ta.key 的方向参数（0/1）配置错误这一经典故障源。

3.5 密钥协商与前向保密

OpenVPN 通过 **DH（Diffie-Hellman）或 ECDH** 完成会话密钥协商。它的妙处在于：双方各自公开交换一段"半成品"，却能算出只有彼此知道的对称密钥，窃听者即使拿到全部交换报文也算不出来。

配合**周期性重协商**（默认每小时或每传输一定数据量后），每次会话使用全新密钥——即使某一次密钥泄露，历史与后续流量依然安全，这就是**前向保密（PFS）**。这是服务端必须准备 `dh.pem` 的原因（使用 ECDH 时可省略）。

3.6 认证方式的三个层次

OpenVPN 支持"证明你拥有什么"与"证明你是谁"的组合认证：

层次	方式	强度	适用
机器认证	客户端证书	高	所有企业部署的基线
用户认证	用户名/密码（PAM、LDAP、RADIUS）	中	与账号体系联动
动态因子	TOTP 动态码 / 短信 / 硬件 Key	最高	高安全要求场景

> **注意**
>
> 证书会随配置文件一起分发，一旦员工笔记本丢失，"证书 + 配置"等于大门钥匙。因此企业级部署应至少启用**证书 + 用户名密码**双重认证，敏感岗位再加 TOTP 动态码（第 9 章实现）。

PART 1 · 基础与原理

第4章　隧道封装与路由原理

> 加密解决"安全"，路由解决"到达"。本章解释 OpenVPN 如何借助 TUN 虚拟网卡与操作系统路由表，把报文从应用程序一路送进隧道，再在另一端还原——这是排障时最高频用到的一章。

4.1　TUN/TAP：操作系统里的"假网卡"

OpenVPN 隧道的两端各有一张由内核提供的**虚拟网卡**：

设备	工作层次	承载单元	OpenVPN 参数
TUN	第 3 层（网络层）	IP 报文	`dev tun` （默认）
TAP	第 2 层（链路层）	以太网帧	`dev tap`

TUN 的特殊性在于：应用程序写入它的 IP 报文，不会真的发上网络，而是被**打开这个设备的用户态进程（OpenVPN）读走**；反过来，OpenVPN 写入 TUN 的报文会被内核当作"刚从网卡收到"来处理。一张小小的虚拟网卡，架起了内核网络栈与用户态加密进程之间的桥梁。

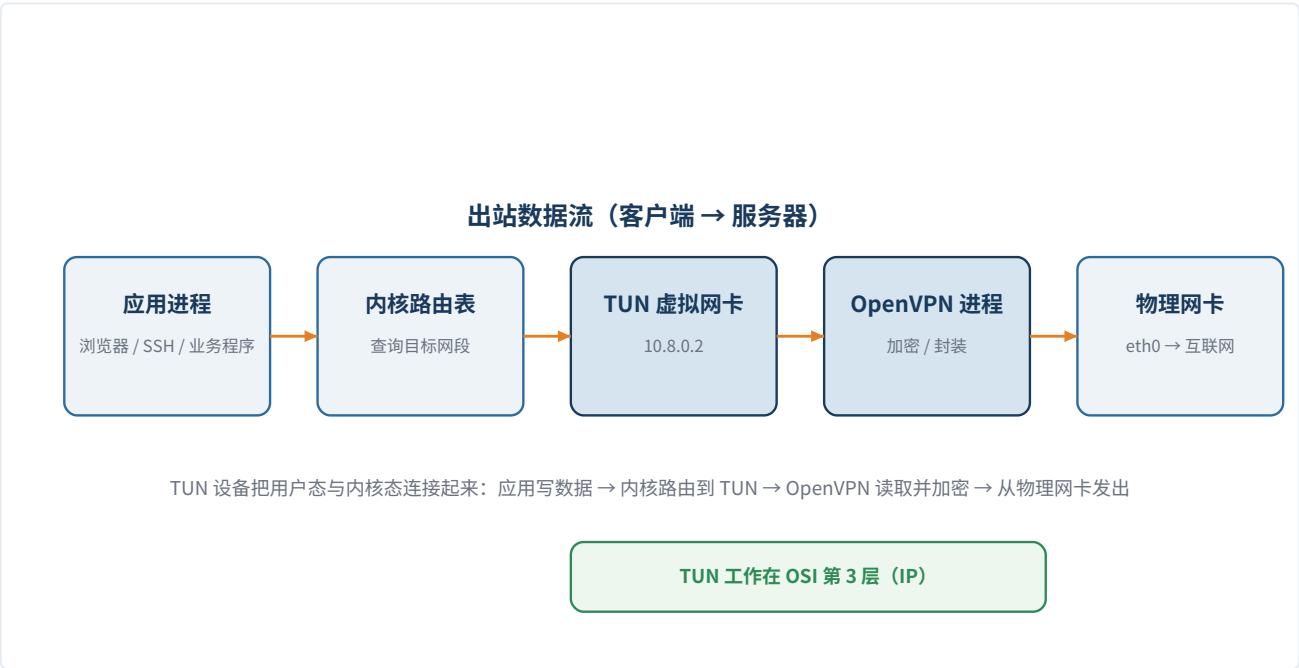


图 4-1　出站数据流：应用 → 路由表 → TUN → OpenVPN 加密 → 物理网卡

TAP 能传递二层广播、支持 Windows 网上邻居等老式协议，但开销更大；除特殊兼容需求外，**一律使用 TUN**。

4.2 封装与解封装：报文的"套娃"

当客户端访问内网服务器（如 192.168.1.10）时，报文的旅程如下：

1. 路由表匹配发现 192.168.1.0/24 需走 `tun0`，报文被送入 TUN 设备；
2. OpenVPN 从 TUN 读出完整 IP 报文，用会话密钥加密并附加认证标签；
3. 加上 OpenVPN 协议头与外层 IP/UDP 头，从物理网卡发往服务器；
4. 服务器解出外层，校验、解密，把内层报文写进自己的 `tun0`，内核按路由将其送达 192.168.1.10。

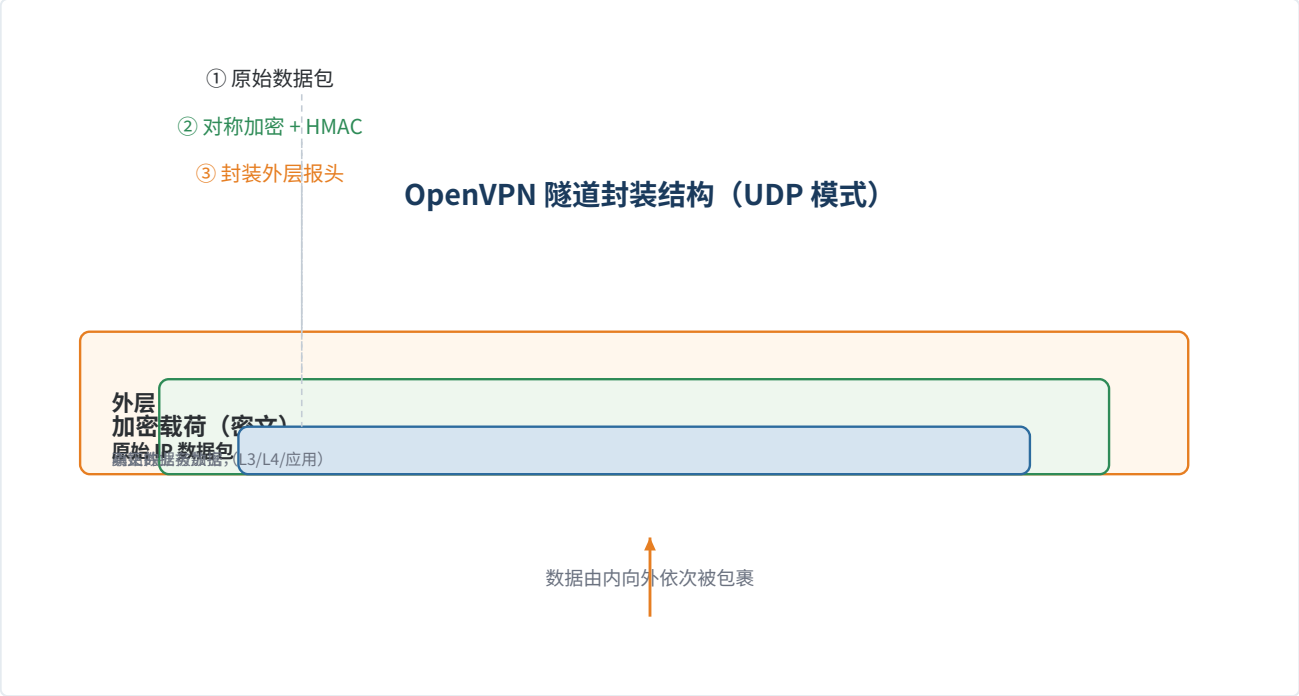


图 4-2 隧道封装结构：原始报文被加密后，再套上外层 IP/UDP 报头在公网传输

4.3 拓扑模式：虚拟 IP 如何编址

服务端参数 `topology` 决定虚拟网段的编址方式，这直接影响 iptables 规则的写法：

模式	地址分配	评价
net30（默认，旧版）	每个客户端消耗 /30 网段（4 个地址）	兼容点对点语义，地址浪费大，规则复杂
subnet（推荐）	所有客户端共享一个网段，按位分配单 IP	直观省地址，iptables/Windows 均友好

> **提示**
>
> 新部署请固定使用 `topology subnet`：服务端拿 10.8.0.1，客户端从 10.8.0.2 起顺序分配，所见即所得。

4.4 路由推送与全局代理

服务端通过 `push` 指令向客户端下发路由，控制"哪些流量走隧道"：

```
# 只让客户端访问公司内网网段（分流模式）
push "route 192.168.1.0 255.255.255.0"
push "route 10.0.0.0 255.0.0.0"

# 让客户端全部流量走 VPN（全局模式）
push "redirect-gateway def1 bypass-dhcp"
```

`redirect-gateway` 的原理是向客户端注入两条更精确的默认路由（0.0.0.0/1 与 128.0.0.0/1），"覆盖"而不破坏原有默认路由——断开 VPN 后原路由自动恢复生效，这正是它优雅的地方。

4.5 让客户端借道访问互联网：NAT

全局模式下，客户端的报文到达服务器后还要"换源地址"才能访问外网，否则回程流量找不到路。这需要在服务器上做 **NAT（伪装）**：

```
iptables -t nat -A POSTROUTING -s 10.8.0.0/24 -o eth0 -j MASQUERADE
```

同时确认内核 IP 转发已打开（`net.ipv4.ip_forward=1`）。这两项配置是"连上 VPN 但上不了网"的头号原因，第 7 章与第 13 章还会反复强调。

PART 2 · 安装与配置

第5章　环境准备与安装

> 从本章起进入动手阶段。我们先把服务器环境准备好：选型、上云、放行端口、安装软件，并给出"一键脚本"与"手动安装"两条路线。

5.1　服务器选型与规格建议

规模	并发用户	建议规格	说明
小型	≤ 50	1 核 1G / 1M 带宽	普通云主机即可
中型	50 ~ 300	2 核 4G / 按需带宽	开启 AES-NI 的 CPU 收益明显
大型	300+	多实例 + 负载均衡	见第 10 章高可用架构

> **提示**
>
> 选择支持 **AES-NI** 指令集的 CPU（`grep -c aes /proc/cpuinfo` 大于 0），AES-GCM 加解密可快数倍，这是性价比最高的"性能优化"。

5.2　云环境准备清单

- **安全组 / 防火墙：**放行 1194/UDP（或你自定义的端口）。云服务器最常见故障就是"本机能通、公网连不上"，十有八九是安全组没放行。
- **弹性公网 IP 或域名：**建议绑定一个域名（如 `vpn.example.com`），日后换服务器只需改 DNS，客户端配置不用变。
- **时间同步：**证书校验依赖准确时间，确认 NTP 已启用。
- **系统：**Ubuntu 22.04 / Debian 12 / Rocky Linux 9 均可，本书示例以 Ubuntu 为准。

5.3　安装 OpenVPN

各主流发行版的安装都很简单：

```
# Debian / Ubuntu
sudo apt update && sudo apt install -y openvpn easy-rsa
```

```
# RHEL / Rocky / CentOS Stream
sudo dnf install -y epel-release
sudo dnf install -y openvpn easy-rsa

# 验证版本（要求 2.5+）
openvpn --version | head -1
```

客户端软件则按平台下载：Windows 用 [OpenVPN GUI](https://openvpn.net/client/)，macOS 用 Tunnelblick，Android/iOS 用 OpenVPN Connect，Linux 直接用软件包管理器安装 openvpn 包。

5.4 路线一：一键脚本（推荐新手）

社区脚本 `angristan/openvpn-install` 把证书、服务端配置、客户端文件生成全部自动化：

```
curl -O https://raw.githubusercontent.com/angristan/openvpn-install/master/openvpn-install.sh
chmod +x openvpn-install.sh
sudo ./openvpn-install.sh
```

脚本会交互式询问公网 IP、端口、协议、DNS 与首个客户端名称，全自动完成 PKI 搭建、服务端配置与首个客户端 `.ovpn` 文件的生成。后续重复运行脚本可添加/吊销用户。

> **说明**
>
> 一键脚本适合快速上手与小型团队。它生成的配置默认较宽松，**企业环境请务必在理解每一项含义后**（第 7、11 章）再决定采用或调整。

5.5 路线二：手动安装（推荐企业）

手动路线 = 第 6 章（PKI）+ 第 7 章（服务端配置）+ 第 8 章（客户端）三章的顺序执行。虽然步骤多一点，但每一块都可审计、可复现，便于纳入 Ansible 等自动化工具，这是企业落地的正道。

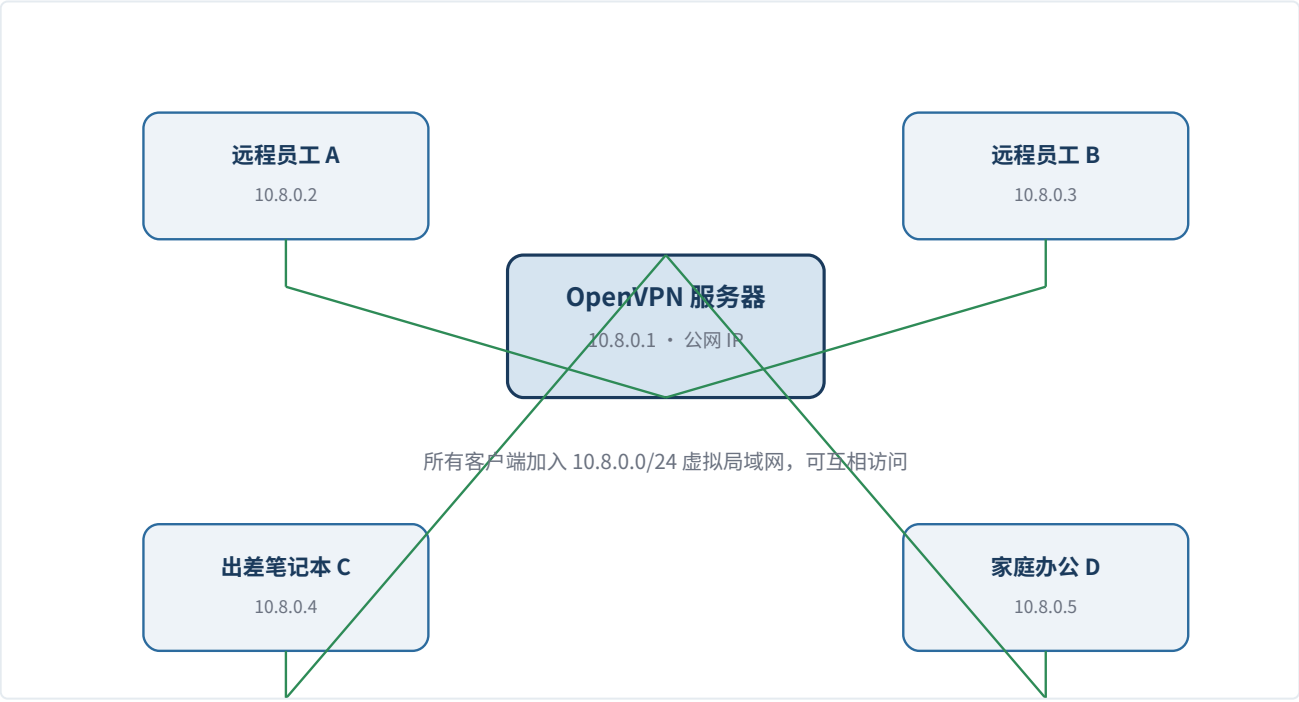


图 5-1　本章要搭建的目标拓扑：一台云上服务器，服务多个远程客户端（10.8.0.0/24）

PART 2 · 安装与配置

第6章　PKI 证书体系搭建

> 证书是 OpenVPN 企业部署的灵魂。本章使用 Easy-RSA 3 完整搭建一套 PKI：根 CA、服务器证书、客户端证书、DH 参数与 TLS 认证密钥，并给出企业级的证书管理规范。

6.1　初始化 Easy-RSA 工作目录

Easy-RSA 是 OpenVPN 官方配套的命令行 CA 工具。建议把 CA 目录放在**独立于 VPN 服务器的机器**上（哪怕是台笔记本），签发完毕后私钥离线保存。

```
# 复制一份 Easy-RSA 工作目录
make-cadir ~/openvpn-ca && cd ~/openvpn-ca

# （可选）编辑 vars 设置默认值，避免每次交互输入
cat >> vars <<'EOF'
set_var EASYRSA_REQ_COUNTRY    "CN"
set_var EASYRSA_REQ_PROVINCE   "Beijing"
set_var EASYRSA_REQ_CITY       "Beijing"
set_var EASYRSA_REQ_ORG        "Example Corp"
set_var EASYRSA_REQ_EMAIL      "admin@example.com"
set_var EASYRSA_REQ_OU         "IT"
set_var EASYRSA_CA_EXPIRE      3650
set_var EASYRSA_CERT_EXPIRE    825
EOF
```

> **说明**
>
> 客户端证书有效期建议不超过 825 天（与主流浏览器/移动端策略对齐），到期前滚动换发。

6.2　构建根 CA

```
./easyrsa init-pki          # 初始化 PKI 目录
./easyrsa build-ca nopass   # 创建根 CA（nopass 表示私钥不加密码）
```

执行完毕后得到两个关键文件：`pki/ca.crt`（证书，分发给所有人）与 `pki/private/ca.key`（私钥，**最高机密**）。

6.3 签发服务器证书

```
./easyrsa gen-req server nopass
./easyrsa sign-req server server      # 用 CA 签署
```

生成 `pki/issued/server.crt` 与 `pki/private/server.key`。

6.4 签发客户端证书

每位用户一张证书，命名规范建议 `部门-姓名` 或工号，例如：

```
./easyrsa gen-req zhangsan nopass
./easyrsa sign-req client zhangsan
```

6.5 DH 参数与 TLS 认证密钥

```
./easyrsa gen-dh                      # 生成 dh.pem（耗时数分钟）
openvpn --genkey secret ta.key        # 生成 tls-auth/tls-crypt 密钥
```

6.6 吊销证书与 CRL

员工离职、设备丢失时，必须能立即吊销其证书：

```
./easyrsa revoke zhangsan
./easyrsa gen-crl          # 重新生成吊销列表 crl.pem
# 将 crl.pem 分发到服务端，并在 server.conf 中确认：
#   crl-verify crl.pem
```

之后该用户的连接会被服务端直接拒绝。**CRL 更新是运维常态，请把"生成并分发 CRL"写进离职流程。**

6.7 企业证书管理规范

事项	规范

CA 私钥	离线保管（U 盘/加密盘），双人操作，禁入 VPN 服务器
证书命名	一证一人一设备，禁止共用；命名含工号便于审计
有效期	客户端 ≤ 825 天；到期前 30 天启动换发
吊销	离职当天吊销 + 更新 CRL，与 HR 流程联动
备份	PKI 目录加密备份；ca.key 单独存放

PART 2 · 安装与配置

第7章　服务端配置详解

> 本章给出一份"企业级可直接使用"的 server.conf，并逐项解释每个参数为什么这样写，最后打通 IP 转发与防火墙——这是从"能连"到"能用"的关键一步。

7.1　企业级 server.conf 完整示例

先把证书文件复制到位，再写配置：

```
sudo mkdir -p /etc/openvpn/server
sudo cp pki/ca.crt pki/issued/server.crt pki/private/server.key pki/dh.pem ta.key pki/crl.pem /etc/
openvpn/server/
```

创建 `/etc/openvpn/server/server.conf`：

```
# ---- 端口与协议 ----
port 1194
proto udp
dev tun
topology subnet

# ---- 证书与密钥 ----
ca /etc/openvpn/server/ca.crt
cert /etc/openvpn/server/server.crt
key /etc/openvpn/server/server.key
dh /etc/openvpn/server/dh.pem
tls-crypt /etc/openvpn/server/ta.key
crl-verify /etc/openvpn/server/crl.pem

# ---- 虚拟网段 ----
server 10.8.0.0 255.255.255.0
ifconfig-pool-persist /var/log/ipp.txt

# ---- 下发给客户端的路由 ----
push "route 192.168.1.0 255.255.255.0"
push "dhcp-option DNS 223.5.5.5"

# ---- 安全与性能 ----
data-ciphers AES-256-GCM:AES-128-GCM
auth SHA256
keepalive 10 120
```

```
user nobody
group nogroup
persist-key
persist-tun

# ---- 日志 ----
status /var/log/openvpn/status.log
log-append /var/log/openvpn/server.log
verb 3
```

7.2 核心参数逐项解析

参数	含义	要点
proto udp	承载协议	性能优先；受限网络换 tcp
topology subnet	编址模式	全端统一，规则直观
tls-crypt	控制信道加密	优于 tls-auth，防探测
crl-verify	证书吊销校验	不配置则吊销无效！
server 10.8.0.0/24	虚拟网段	避免与办公网/家庭网重叠
push route	下发路由	按需推送，最小权限
keepalive	保活探测	10 秒 ping、120 秒判死
user/group nobody	降权运行	初始化后放弃 root 权限
verb 3	日志级别	生产 3 足够，排障临时开 4-5

7.3 IP 转发与防火墙

```
# 1. 开启内核转发
echo 'net.ipv4.ip_forward=1' | sudo tee -a /etc/sysctl.conf
sudo sysctl -p

# 2. NAT：让虚拟网段可以借道访问服务器所在网络
sudo iptables -t nat -A POSTROUTING -s 10.8.0.0/24 -o eth0 -j MASQUERADE

# 3. 放行 OpenVPN 端口与转发链
sudo ufw allow 1194/udp

# 4. 持久化 iptables 规则（重启不丢）
```

```
sudo apt install -y iptables-persistent
sudo netfilter-persistent save
```

> **注意**
>
> 网卡名 `eth0` 请按实际替换（`ip route` 查看默认路由出口）。云服务器还需在**安全组**中放行 1194/UDP——这是新手部署失败率最高的一个环节。

7.4 启动与自检

```
sudo systemctl enable --now openvpn-server@server
systemctl status openvpn-server@server
ss -lnup | grep 1194          # 确认端口在监听
sudo openvpn --config /etc/openvpn/server/server.conf --test-crypto  # 语法自检
```

看到 `Initialization Sequence Completed` 或端口监听正常，服务端即就绪。

PART 2 · 安装与配置

第8章　客户端配置详解

> 一个 .ovpn 文件就是用户的"钥匙"。本章讲清客户端配置的每一项、如何生成自带证书的单文件配置、各平台客户端怎么用，以及配置文件的安全分发方式。

8.1　client.ovpn 逐项解析

```
client                          # 声明这是客户端
dev tun
proto udp
remote vpn.example.com 1194     # 服务端地址与端口（建议用域名）
resolv-retry infinite           # 域名解析失败时无限重试
nobind                          # 不绑定固定本地端口
persist-key
persist-tun
remote-cert-tls server          # 关键：校验对方确实是"服务端角色"
tls-crypt ta.key
data-ciphers AES-256-GCM:AES-128-GCM
verb 3
```

`remote-cert-tls server` 防止"恶意服务端用一张客户端证书伪装"，属于必配项。证书部分用标签内嵌：

```
<ca>
-----BEGIN CERTIFICATE-----
... (ca.crt 内容)
-----END CERTIFICATE-----
</ca>
<cert> ... client 证书 ... </cert>
<key> ... client 私钥 ... </key>
<tls-crypt> ... ta.key 内容 ... </tls-crypt>
```

8.2　一键生成内嵌式单文件配置

手动拼证书容易出错，用脚本自动合并：

```
#!/bin/bash
# make_client.sh <客户端名>：生成自带证书的 .ovpn
NAME=$1; CA=~/openvpn-ca/pki
```

```
cat > ${NAME}.ovpn <<EOF
client
dev tun
proto udp
remote vpn.example.com 1194
resolv-retry infinite
nobind
persist-key
persist-tun
remote-cert-tls server
tls-crypt ta.key
data-ciphers AES-256-GCM:AES-128-GCM
verb 3
<ca>$(cat ${CA}/ca.crt)</ca>
<cert>$(openssl x509 -in ${CA}/issued/${NAME}.crt)</cert>
<key>$(cat ${CA}/private/${NAME}.key)</key>
<tls-crypt>$(cat ta.key)</tls-crypt>
EOF
echo "已生成 ${NAME}.ovpn"
```

8.3 各平台客户端

平台	软件	操作
Windows	OpenVPN GUI	导入 .ovpn → 托盘图标右键连接
macOS	Tunnelblick	双击 .ovpn 安装，菜单栏连接
Linux	openvpn 命令行 / NetworkManager	`sudo openvpn --config user.ovpn`
Android / iOS	OpenVPN Connect	扫码或导入文件后连接

8.4 连接验证清单

1. 客户端日志出现 `Initialization Sequence Completed`；
2. `ip addr show tun0`（Windows: `ipconfig`）拿到 10.8.0.x 地址；
3. `ping 10.8.0.1` 通（隧道层 OK）；
4. `ping 192.168.1.10` 通（内网路由 OK）；
5. 服务端 `status.log` 能看到该用户的连接记录。

8.5 配置文件的安全分发

> **注意**
>
> .ovpn 文件内含用户私钥，等同账号密码。**禁止**通过微信群、邮件明文群发。推荐方式：内部 HTTPS 门户登录后下载（绑定工号）、二维码现场扫码、或由 IT 面对面部署。

PART 3 · 进阶与企业实践

第9章　高级特性：双因素认证与脚本钩子

> OpenVPN 的真正威力在于可编程。本章覆盖企业落地最常用的四件事：双因素认证、按用户定制的 ccd 机制、脚本钩子体系与细粒度的推送策略。

9.1　证书 + TOTP：双因素认证实战

借助 PAM 插件与 Google Authenticator，实现"证书 + 动态码"双因素：

```
# 1. 安装
sudo apt install -y libpam-google-authenticator openvpn-auth-ldap 2>/dev/null || \
sudo apt install -y libpam-google-authenticator

# 2. 为用户生成 TOTP 密钥（用户手机扫码绑定）
google-authenticator -t -d -f -r 3 -R 30 -w 3

# 3. /etc/pam.d/openvpn 追加：
auth required pam_google_authenticator.so

# 4. server.conf 追加：
plugin /usr/lib/x86_64-linux-gnu/openvpn/plugins/openvpn-plugin-auth-pam.so openvpn
verify-client-cert require
```

客户端配置追加 `auth-user-pass`，连接时输入"密码 + 6 位动态码"。此后即使证书泄露，没有动态码也无法登录。

9.2　ccd：按用户分配静态 IP 与专属路由

启用客户端配置目录后，每个证书可拥有一份专属配置：

```
# server.conf
client-config-dir /etc/openvpn/ccd

# /etc/openvpn/ccd/zhangsan
ifconfig-push 10.8.0.50 255.255.255.0   # 固定虚拟 IP
push "route 10.200.0.0 255.255.255.0"   # 额外推送生产网段
```

文件名必须与证书 CN 完全一致。这是"按人授权"最简单可靠的实现。

9.3 脚本钩子：接入自动化体系

OpenVPN 在连接生命周期的关键节点会调用外部脚本，钩子一览：

钩子	触发时机	典型用途
auth-user-pass-verify	用户名密码验证	对接 LDAP/RADIUS/数据库
client-connect	客户端上线	记录审计、动态下发路由
client-disconnect	客户端下线	清理防火墙规则、统计时长
learn-address	虚拟 MAC/IP 变化	站点互联时维护转发规则

例如上线审计脚本：

```
# /etc/openvpn/scripts/on-connect.sh
#!/bin/bash
echo "$(date '+%F %T') user=$common_name ip=$trusted_ip vip=$ifconfig_pool_remote_ip" \
  >> /var/log/openvpn/access.log
```

9.4 推送 DNS 与分裂隧道

```
push "dhcp-option DNS 10.8.0.1"          # 用内网 DNS 解析内网域名
push "register-dns"                       # Windows 生效
# 分裂隧道：只代理公司网段，其余流量走本地网络（节省带宽、体验更好）
push "route 192.168.0.0 255.255.0.0"
```

9.5 性能相关选项

> **提示**
>
> 现代版本开启 AES-NI 后加解密已不是瓶颈。关于压缩：**不要开启 comp-lzo/compress**——收益微小，且 CRIME/BREACH 类攻击面增加，OpenVPN 官方同样建议关闭。

PART 3 · 进阶与企业实践

第10章　企业高可用与负载均衡

> 单点服务器撑不起企业的可用性承诺。本章给出三种由轻到重的高可用方案，并讲清多实例部署中配置与证书的统一管理问题。

10.1　高可用架构总览

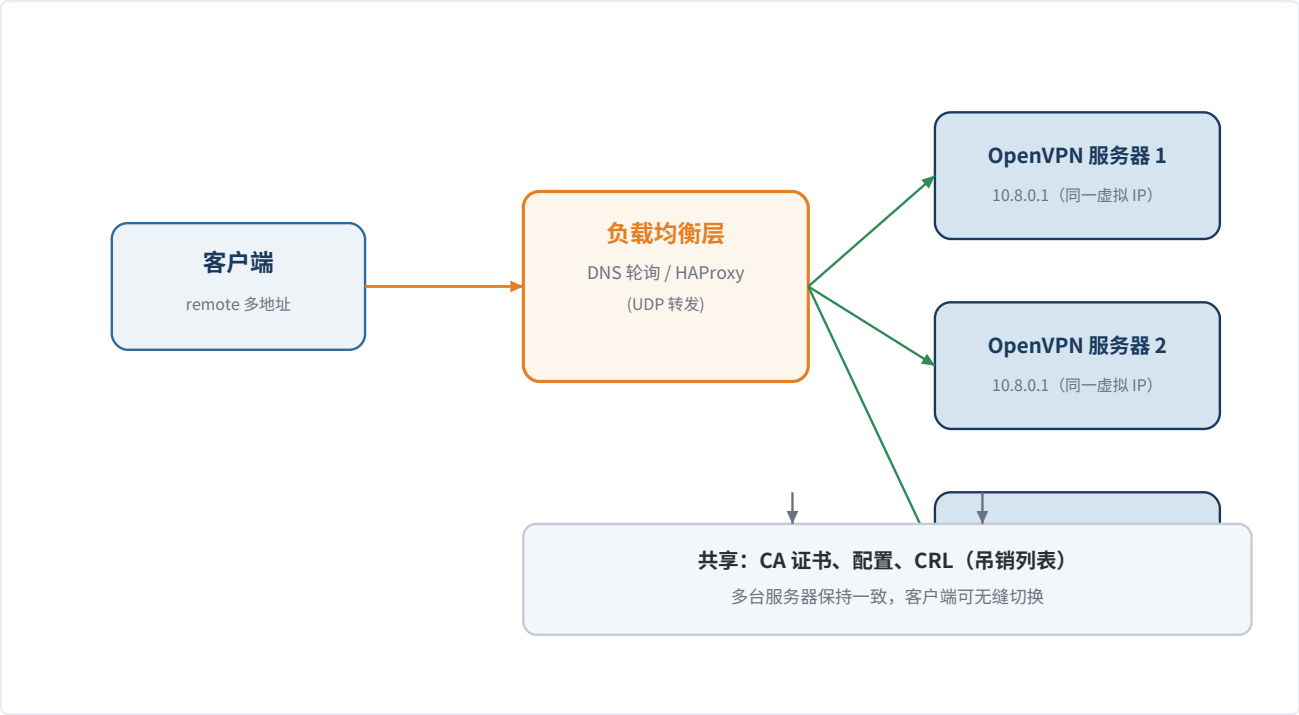


图 10-1　多实例 + 前置负载均衡的高可用架构，后端共享同一套 PKI

要点：所有实例使用**同一 CA 签发的证书**、相同的虚拟网段配置，客户端可以随意切换实例而无需感知。

10.2　方案一：客户端多地址故障切换（零成本）

```
remote vpn1.example.com 1194
remote vpn2.example.com 1194
remote-random            # 在多个地址间随机选择，天然分流
connect-retry-max 3      # 单地址重试次数后切下一个
```

无需任何额外组件，适合中小规模。**这是性价比最高的第一道保险。**

10.3 方案二：DNS 轮询

给 `vpn.example.com` 配多条 A 记录指向不同服务器，客户端天然分散到各实例。缺点是故障收敛依赖 DNS 缓存时间，建议 TTL 控制在 60 秒内。

10.4 方案三：HAProxy UDP 负载均衡

```
# /etc/haproxy/haproxy.cfg（1.9+ 支持 UDP）
listen openvpn-udp
  mode tcp
  bind *:1194 udp@4
  balance roundrobin
  timeout client  60s
  timeout server  60s
  server vpn1 10.0.1.11:1194 check
  server vpn2 10.0.1.12:1194 check
  server vpn3 10.0.1.13:1194 check
```

配合健康检查，坏节点自动摘除。注意会话保持：OpenVPN 客户端与同一实例保持长连接，HAProxy 按 `balance source` 也可保证同一源 IP 固定到同一后端。

10.5 配置与证书的统一管理

多实例下最怕"配置漂移"。推荐做法：

- **Ansible/Salt 统一下发** server.conf 与 crl.pem，Git 仓库作为唯一事实来源；
- 证书签发集中在 CA 主机，签发后经加密通道分发；
- CRL 更新后批量同步到所有实例并 `systemctl reload openvpn-server@server`。

10.6 站点到站点互联的高可用



图 10-2 总部与分支通过网关互联，两站点内网互达

站点互联使用 `mode server` 的对等变体：两端各起一个 P2P 隧道，互为客户端与服务器，配合 `iroute` 声明"哪个网段在隧道对端"。跨站点高可用可建立两条隧道（走不同运营商线路）+ 动态路由（如 Bird/OSPF）自动切换。

10.7 证书轮换不停机

1. 新证书提前签发、随配置更新下发；
2. 设置 `reneg-sec` 让会话定期重协商；
3. 旧证书到期前吊销并更新 CRL，全实例同步。

PART 3 · 进阶与企业实践

第11章　安全加固最佳实践

> "能跑起来"与"敢用于生产"之间隔着一整章的距离。本章按证书、网络、系统、算法、审计五个维度给出加固清单，每一条都可直接落地。

11.1　证书体系加固

- CA 私钥离线保管，签发操作双人复核；
- 一证一人一设备，禁止共用；证书 CN 命名可追溯；
- 强制 `crl-verify`，吊销流程与 HR 离职流程联动；
- 启用 `tls-crypt` 替代裸 TLS 握手，隐藏服务指纹；
- 证书 + 用户名密码（或 TOTP）双因素，见第 9 章。

11.2　网络层加固

```
# 1) 端口改为高位并限速防扫描（示例）
port 51994

# 2) fail2ban 自动封禁反复认证失败的来源 IP
# /etc/fail2ban/jail.local
[openvpn]
enabled  = true
port     = 51994
protocol = udp
filter   = openvpn
logpath  = /var/log/openvpn/server.log
maxretry = 5
findtime = 600
bantime  = 86400
```

- iptables 仅放行必须的网段访问内网，默认拒绝；
- 虚拟网段避免使用 `192.168.1.0/24` 这类与家庭路由器重叠的网段（防止路由冲突）；
- 管理面（SSH、监控）不暴露公网，可只允许从 VPN 网段访问。

11.3 系统与进程加固

- `user nobody / group nogroup` 降权 + `chroot`（可选）；
- 保持 OpenVPN 版本跟进（关注官方 CVE 公告）；
- 服务器最小化安装，关闭无关服务；
- 配置与证书目录权限收紧：`chmod 600 /etc/openvpn/server/*.key`。

11.4 算法与协议加固

项	要求
数据加密	AES-256-GCM / AES-128-GCM / ChaCha20-Poly1305
禁用项	BF-CBC、DES、RC2、SHA1 认证
TLS 版本	`tls-version-min 1.2`（推荐 1.3）
认证摘要	SHA256 及以上

11.5 审计与合规

- 开启 `client-connect/disconnect` 审计日志：谁、何时、从哪个 IP、拿到什么地址；
- 日志集中收集（rsyslog → 日志平台），保留期满足等保/内审要求（一般 ≥ 6 个月）；
- 定期导出活跃证书清单，与在职人员表核对，清理"僵尸证书"；
- 重大变更（网段调整、算法变更）走变更管理流程。

11.6 上线前安全检查清单

#	检查项	标准
1	ca.key 是否离线	服务器上不存在 ca.key
2	CRL 是否启用并有效	吊销测试通过
3	双因素是否启用	无动态码无法登录
4	弱算法是否禁用	data-ciphers 无 CBC/SHA1
5	日志与审计是否落地	能回答"谁在何时连入"

6	防火墙默认策略	默认拒绝，最小放行

PART 3 · 进阶与企业实践

第12章　监控、日志与告警

> 没有监控的 VPN 服务等于"薛定谔的可用性"。本章搭起一套 Prometheus + Grafana 的标准监控体系，并给出企业最该盯的指标与告警阈值。

12.1　监控体系架构

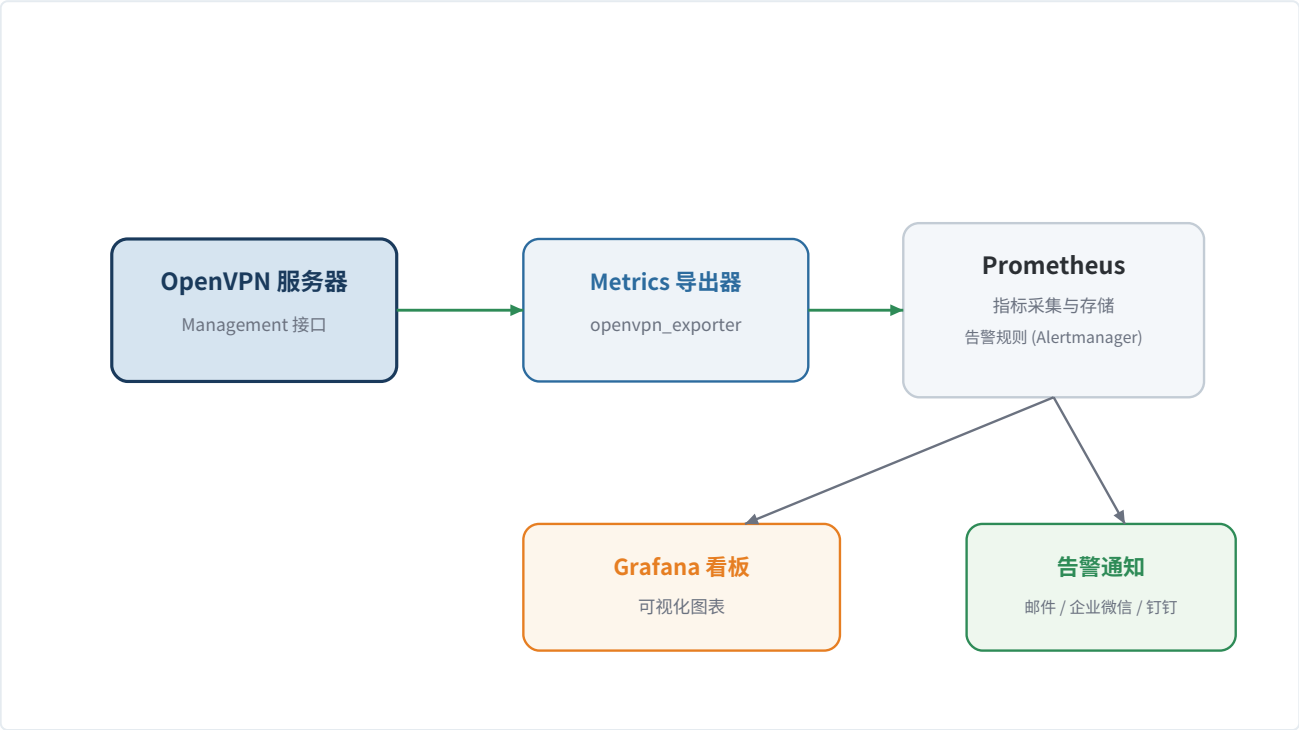


图 12-1　OpenVPN 监控链路：management 接口 → exporter → Prometheus → Grafana / 告警

12.2　打开 management 接口

```
# server.conf 追加（仅监听本机，避免暴露）
management 127.0.0.1 7505 /etc/openvpn/server/mgmt.pass
management-client
```

验证：

```
telnet 127.0.0.1 7505
> status     # 输出当前所有客户端连接表
> quit
```

12.3 接入 Prometheus + Grafana

```
# 部署 openvpn_exporter（以 Docker 为例）
docker run -d --name openvpn_exporter \
  -p 9176:9176 \
  kumina/openvpn-exporter \
  -openvpn.status_paths /etc/openvpn/status.log
```

Prometheus 抓取配置：

```
scrape_configs:
  - job_name: openvpn
    static_configs:
      - targets: ["10.0.1.11:9176"]
```

Grafana 导入社区 OpenVPN 看板（Dashboard ID：10699 等），即可获得在线人数、流量、每用户会话等图表。

12.4 关键监控指标

指标	来源	意义
在线用户数	status.log	容量与趋势规划
每用户流量	status.log	异常大流量排查（盗用/滥用）
进程存活	node_exporter	服务可用性
CPU / 内存	node_exporter	加密开销观测
认证失败次数	日志关键字	暴力破解探测

12.5 告警策略建议

- **P1（5 分钟响应）**：服务进程消失、端口不可达；

- **P2（1 小时内）：** 单实例认证失败突增（>20 次/分）、CPU 持续 >85%；
- **P3（次日处理）：** 在线数异常波动、证书 30 天内到期清单。

> **提示**
>
> 把"证书到期"做成自动巡检：用 openssl 批量读取证书 notAfter，距离到期 30 天时自动邮件提醒用户换发，可消除绝大多数"突然连不上"的工单。

PART 3 · 进阶与企业实践

第13章　故障排查实战

> VPN 故障 90% 落在五类：网络不通、配置笔误、证书问题、防火墙拦截、MTU 黑洞。本章给出一套自顶向下的排查方法论与高频问题速查表。

13.1　排查方法论：分层定位

1. **网络层：** `ping 公网IP`、`telnet host 1194` 能否到达？
2. **传输层：** UDP 还是被防火墙静默丢弃？服务器 `tcpdump` 有无收到包？
3. **TLS 层：** 日志是否出现证书验证失败？
4. **隧道层：** 握手完成但 ping 不通 10.8.0.1？
5. **路由/NAT 层：** 能 ping 10.8.0.1 但到不了内网？

先把故障"钉"在某一层，再在该层内找证据，比盲目重装高效十倍。

13.2　高频故障速查表

现象	常见原因	解决
TLS Error: TLS key negotiation failed	端口未放行 / proto 不匹配 / tls-auth 方向错	查安全组与两端协议；tls-crypt 替代 tls-auth
Certificate verification failed	缺 ca、时间不同步、证书过期	核对 ca.crt；`timedatectl set-ntp true`
AUTH_FAILED	用户名密码/TOTP 错误；PAM 配置错	核对凭据与 /etc/pam.d/openvpn
连上但 ping 不通 10.8.0.1	两端网段冲突 / top-ology 不一致	统一 topology subnet，更换虚拟网段
ping 通 VPN 但进不了内网	未开 ip_forward / 缺 NAT / 缺 push route	按 4.5 节与 7.3 节补齐
能连但网页打不开、大包丢失	MTU 黑洞	见 13.4 节

13.3 日志分析要点

```
# 服务端实时日志
sudo tail -f /var/log/openvpn/server.log
```

重点关键字：`VERIFY OK`（证书通过）、`Peer Connection Initiated`（会话建立）、`VERIFY ERROR`（证书问题，附原因）、`TLS Error`（握手失败）、`scheduled exit`（服务端主动断开）。

13.4 MTU 黑洞：最隐蔽的问题

症状极具迷惑性：ping 小包全通、SSH 能登录，但网页打不开、文件传不动。原因是隧道层层封装后报文超过链路 MTU，且被 DF 位禁止分片，大包被静默丢弃。

```
# 服务端与客户端同时下调
tun-mtu 1400
mssfix 1360
```

定位技巧：用 `ping -M do -s 1372 目标` 逐档测试，找出可通过的最大包长。

13.5 抓包三板斧

```
# 1) 公网侧：确认客户端报文到达
sudo tcpdump -ni eth0 udp port 1194

# 2) 隧道侧：确认解封装后的流量
sudo tcpdump -ni tun0

# 3) 内网侧：确认转发到目标
sudo tcpdump -ni eth0 host 192.168.1.10
```

三段抓包，报文断在哪一段，问题就在哪一段——这是网络排障的"通用公式"。

PART 3 · 进阶与企业实践

第14章　企业实战案例

> 把前十三章的知识装进四个真实感极强的企业场景。每个案例都给出需求、架构、关键配置与落地要点，可直接作为你所在公司的方案蓝本。

14.1　案例一：50 人公司远程办公接入

需求：员工在家可访问内网 OA 与代码库；离职立即收权；成本最小。

方案要点：

- 一台 2 核 4G 云主机，Ubuntu 22.04，OpenVPN 2.5+；
- 证书 + 用户名密码双因素；分裂隧道（只推送办公网段路由），节省带宽；
- 客户端 remote 配置主备两个域名（主实例 + 备用冷备）；
- 人事离职 → 当天吊销证书 + 更新 CRL（写进离职 checklist）。

核心配置差异：

```
server 10.28.0.0 255.255.255.0    # 避开家庭网段 192.168.x
push "route 10.100.0.0 255.255.0.0"   # 仅办公网段
tls-crypt ta.key
plugin .../openvpn-plugin-auth-pam.so openvpn
```

14.2　案例二：多云 / 多机房站点互联

需求：阿里云、腾讯云与本地 IDC 三地内网互通，公网传输加密。

方案要点：选网络最稳定的 IDC 作为中心节点，其余两站点以 site-to-site 客户端方式接入；每站点网关声明各自的 `iroute`：

```
# 中心节点 ccd/alibaba
iroute 172.16.0.0 255.255.0.0     # 阿里云 VPC 网段
# ccd/tencent
iroute 172.17.0.0 255.255.0.0     # 腾讯云 VPC 网段
```

配合各站点的 `push route` / 静态路由，实现三网互达。两条以上运营商线路互备，动态路由协议（OSPF/Bird）实现故障自动切换。

14.3 案例三：分级权限与网络隔离

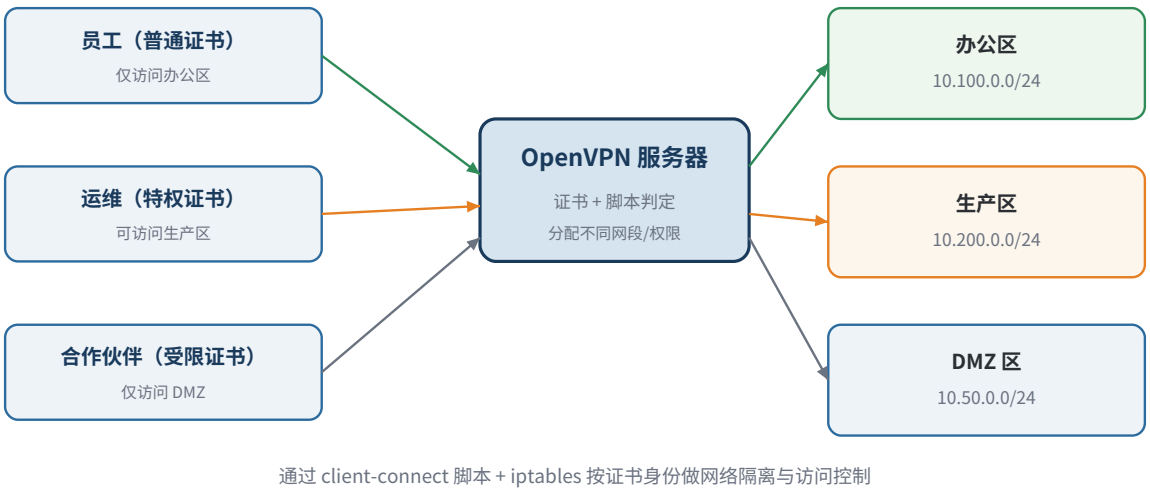


图 14-1 按证书身份划分网络可达范围：办公区、生产区、DMZ 各自隔离

方案要点：证书 CN 命名编码部门与级别（如 `ops-zhangsan`、`partner-wangwu`），通过 ccd + client-connect 脚本把不同用户划入不同虚拟网段，再由 iptables 按网段控制可达区域：

```
# 运维组可访问生产区
iptables -A FORWARD -s 10.8.8.0/24 -d 10.200.0.0/16 -j ACCEPT
# 普通员工仅可达办公区
iptables -A FORWARD -s 10.8.1.0/24 -d 10.100.0.0/16 -j ACCEPT
iptables -A FORWARD -s 10.8.1.0/24 -j REJECT
```

14.4 案例四：等保合规下的审计落地

需求：满足等保对"访问控制 + 安全审计"的要求，事后能完整还原接入行为。

方案要点：

- client-connect/disconnect 双钩子记录：用户、来源 IP、虚拟 IP、起止时间；
- 日志经 rsyslog 转发至集中日志平台，保留 6 个月以上，防篡改；

- 证书清单每月与在职名单核对；
- 关键操作（签发、吊销）双人复核并留痕。

14.5 企业落地路线图

阶段	工作	产出
第 1 周	PoC：单实例跑通，验证网络与性能	测试报告
第 2 周	PKI 规范 + 双因素 + 防火墙基线	安全基线文档
第 3 周	监控告警 + 审计日志接入	运维看板
第 4 周	灰度 10 人 → 全员推广；文档与工单流程	正式上线

> **提示**
>
> 先灰度再推广、把"收权"写进人事流程、把"证书到期"做成自动提醒——这三件事决定了 OpenVPN 企业部署的口碑。

附录

附录A　配置参数速查表

服务端常用参数

参数	示例	说明
port / proto	1194 / udp	监听端口与协议
dev / topology	tun / subnet	虚拟设备与编址模式
server	10.8.0.0 255.255.255.0	虚拟网段与地址池
ca / cert / key	路径	CA、服务器证书与私钥
dh	dh.pem	DH 参数（ECDH 可省）
tls-crypt	ta.key	控制信道加密认证
crl-verify	crl.pem	证书吊销校验
push	"route x.x.x.x mask"	下发路由 / DNS
client-config-dir	/etc/openvpn/ccd	按用户定制
keepalive	10 120	保活间隔与判死时间
reneg-sec	3600	密钥重协商周期
data-ciphers	AES-256-GCM:AES-128-GCM	数据加密算法白名单
user / group	nobody / nogroup	降权运行
status / log-append	路径	状态与日志文件
verb	3	日志级别（排障 4-5）

客户端常用参数

参数	示例	说明
client	—	客户端模式声明
remote	vpn.example.com 1194	服务端地址（可多条）
remote-random	—	多地址随机/分流

remote-cert-tls	server	校验服务端角色
auth-user-pass	—	启用用户名密码/TOTP
tun-mtu / mssfix	1400 / 1360	MTU 调优
pull-filter	ignore "redirect-gateway"	拒绝服务端全局代理
inline 标签	<ca> <cert> <key>	内嵌证书单文件分发

附录

附录B　常用命令速查

服务端运维

```
systemctl start openvpn-server@server      # 启动
systemctl status openvpn-server@server     # 状态
journalctl -u openvpn-server@server -f     # 跟踪日志
ss -lnup | grep 1194                        # 确认端口监听
openvpn --version                           # 查看版本
```

Easy-RSA 证书操作

```
./easyrsa build-ca nopass                  # 创建 CA
./easyrsa gen-req NAME nopass              # 生成请求
./easyrsa sign-req client NAME             # 签发客户端证书
./easyrsa revoke NAME                      # 吊销
./easyrsa gen-crl                          # 更新 CRL
openssl x509 -in cert.crt -noout -dates    # 查看证书有效期
```

连接与诊断

```
sudo openvpn --config client.ovpn          # 命令行连接
ip addr show tun0                          # 查看虚拟网卡
ping 10.8.0.1                              # 隧道连通性
traceroute 192.168.1.10                    # 路由路径
sudo tcpdump -ni eth0 udp port 1194        # 抓包
ping -M do -s 1372 10.8.0.1                # MTU 探测
```

防火墙与转发

```
sysctl net.ipv4.ip_forward                 # 查看转发开关
iptables -t nat -L POSTROUTING -n -v       # 查看 NAT 规则
```

```
sudo ufw allow 1194/udp                    # 放行端口
netfilter-persistent save                  # 持久化规则
```

—— 全书完 ——